

Socle industriel et innovation

Détection de fraude de trajectoire par apprentissage automatique,
couplée à la cybersécurité

Argumentaire vérifiable – SwiftDeliver

Projet SwiftDeliver – plateforme de livraison

3 octobre 2026

Comment lire ce document. Il a deux parties : le socle attendu par la grille, puis l'innovation. Chaque affirmation porte un statut : **[F]** fait et vérifié par un test ou un script exécutable ; **[P]** prévu, non implémenté à ce jour ; **[L]** limite connue et assumée. Aucune ligne n'est **[F]** sans un test reproductible, et la commande qui le reproduit est donnée. C'est la règle qui rend ce document utile : il peut être contredit en l'exécutant.

1 Ce que la grille demande

Le socle attendu est l'*industrialisation* (Docker, CI/CD, Cloud public), la *sécurité standard* (JWT, RBAC, variables d'environnement masquées, hachage des mots de passe, validation des entrées) et la *robustesse* (refactorisation de l'existant, UX sans plantage). L'innovation doit *transformer l'usage du produit par un composant d'intelligence artificielle*, couplé au domaine technique secondaire.

2 Socle industriel – livré

Axe	Ce qui est livré	Preuve exécutable
Docker	Deux images multi-étages (backend/Dockerfile, frontend/Dockerfile); <code>docker-compose.yml</code> déclare cinq services (backend, frontend, PostgreSQL/pgvector, Redis, OSRM) avec points de santé et démarrage ordonné.	<code>docker compose up -d --build</code> puis <code>docker compose ps</code> : les cinq services passent <i>healthy</i> .
CI/CD	<code>.github/workflows/ci.yml</code> , trois tâches : tests et packaging du backend, lint et build du frontend, et une tâche <i>images</i> qui construit les deux images et y vérifie des propriétés réelles (modèle d'IA présent, refus de démarrer sans clé de signature, connexion à une vraie base, endpoint de santé).	La tâche <i>images</i> échoue si le modèle disparaît de l'image : c'est elle, et non la compilation, qui prouve que la plateforme est livrable.
Cloud public	<code>render.yaml</code> : blueprint Render en offre gratuite (PostgreSQL 15, Key Value, deux services web Docker, région Francfort), <i>appliqué</i> . <code>DEPLOY.md</code> décrit la procédure et les limites de l'offre.	<code>python scripts/check_render_blueprint.py</code> vérifie en CI la cohérence interne (chaque variable pointe vers un service déclaré). En ligne : <code>https://swiftdeliver-frontend.onrender.com</code> sert la plateforme, sa santé renvoie <code>fraudModel:loaded, fraudTrees:60</code> , et le durcissement y est revérifié (déconnexion→200, jeton révoqué→403, 6 ^e échec→429). Schéma complet : <code>python scripts/validate_render_blueprint.py</code> <code>render.yaml</code> .
Sécurité standard	JWT HS256 dont la clé <i>n'a aucune valeur par défaut</i> (l'application refuse de démarrer sans elle); RBAC par préfixe de route (<code>SecurityConfig</code>); secrets uniquement en variables d'environnement (<code>.env</code> ignoré, <code>.env.example</code> fourni); mots de passe hachés en BCrypt; validation Bean des entrées; réponses construites en DTO, jamais l'entité <code>User</code> ; déconnexion qui révoque côté serveur (<code>tokenVersion</code> incrémenté, claim <code>tv</code> vérifié à chaque requête); limitation de débit des connexions échouées (5 par compte, 50 par adresse).	<code>JwtServiceTest</code> (dont la révocation par <code>tokenVersion</code>); <code>LoginRateLimiterTest</code> (9 cas de politique); <code>HealthEndpointTest</code> vérifie que <i>seul</i> <code>/api/v1/health</code> est public; la CI rejoue dans le conteneur la déconnexion (jeton refusé après) et le 429 après cinq échecs; un conteneur sans clé de signature refuse de démarrer.
Robustesse	<code>GlobalExceptionHandler</code> sépare les erreurs du client des pannes serveur : validation, corps illisible, paramètre manquant et type de chemin incorrect renvoient 400; route inconnue, 404; verbe incorrect, 405; dépendance aval indisponible, 503.	<code>MalformedRequestTest</code> (4 cas); <code>verify_pages.js</code> et <code>verify_stats.js</code> vérifient qu'aucune page ne présente de texte de plantage ni d'erreur non capturée.

Statut du déploiement public. Le blueprint est appliqué sur Render : <https://swiftdeliver-frontend.onrender.com> sert la plateforme, et la santé publique confirme le modèle chargé dans le processus distant (`fraudModel:loaded`, `fraudTrees:60`). Le durcissement d'authentification y a été révérifié : déconnexion→200, ancien jeton→403, 6^e échec→429. Limites de l'offre gratuite (base supprimée après 30 jours, mise en veille) documentées dans `DEPLOY.md`.

3 Innovation : détection de fraude par IA, couplée à la cybersécurité

3.1 Le problème, et pourquoi une règle ne suffit pas

Le GPS d'un téléphone est *déclaratif* : un livreur peut signaler une position qu'il n'occupe pas. Deux conséquences pour la plateforme. D'abord, une livraison peut être « confirmée » depuis l'autre bout de la ville. Ensuite, et c'est le point intéressant, la fraude logistique n'est pas une attaque réseau : c'est un *comportement* dans le temps. Un contrôle naïf (distance > 500 m → alerte) arrive trop tard, quand la livraison est déjà terminée, et il crie au loup pour un livreur bloqué dans le trafic à 600 m de sa cible.

3.2 Ce que fait le composant dans le produit [F]

À chaque diffusion de position, le backend calcule huit variables de mouvement et les note avec une forêt aléatoire. Au-delà d'un seuil, une alerte est créée et affichée dans le tableau de bord administrateur, avec la courbe de score de la livraison. Une alerte *ne bloque pas* le livreur : elle le signale à l'humain qui décide. Le client, lui, ne voit pas la suspicion du modèle : il voit que sa preuve de livraison est en cours de vérification, ce qui est l'information dont il a besoin.

Les huit variables sont : `speed_mps`, `accel_mps2`, `dist_to_target_m`, `progress_ratio`, `heading_change_deg`, `target_bearing_err`, `stall_seconds`, `progress_rate`. Elles sont définies dans `ml/train_trajectory_anomaly` et *recalculées à l'identique* en Java : un test de parité compare 200 vecteurs aux probabilités de scikit-learn, avec une tolérance de 10^{-6} .

3.3 Formation hors ligne, inférence dans le produit [F]

Le modèle est entraîné hors ligne par `ml/train_fraud_model.py`, exporté en JSON compressé (316 Ko, 60 arbres de profondeur 12), et chargé au démarrage par le backend Java qui l'évalue en mémoire. Ce découpage est délibéré : une seule JVM, un seul artefact à déployer. Un microservice d'inférence séparé doublerait la surface à exploiter pour un modèle de 316 Ko. Le chargement du modèle est obligatoire : si le fichier manque ou ne se parse pas, l'application refuse de démarrer, et l'endpoint de santé indique le nombre d'arbres effectivement chargés.

3.4 Pourquoi une forêt aléatoire supervisée, et pas une Isolation Forest

La première version était une *Isolation Forest*, non supervisée. Elle a été construite, mesurée, puis *rejetée* : au même budget de fausses alertes, elle ne fait pas mieux qu'un simple seuil `vitesse > 25 m/s`. Un modèle qu'une règle égale n'est pas une innovation. Quatre détecteurs, à 2 % de fausses alertes sur les mêmes données de test :

Détecteur	Précision	Rappel	<code>drift_then_vanish</code>
Règle de vitesse (> 25 m/s)	0.585	0.358	35.8 %
Isolation Forest	0.585	0.357	35.7 %
Régression logistique	0.602	0.342	34.2 %
Forêt aléatoire	0.780	0.896	89.6 %

La colonne *drift_then_vanish* est le scénario qui décide : une approche d'apparence normale qui s'arrête, puis disparaît. La règle de vitesse le voit à 35,8 %, la forêt à 89,6 %. Deux raisons à

ce résultat. D'une part, séparer un livreur garé d'un livreur à un feu rouge exige une interaction (« toujours loin de la cible *et* plus rien ne bouge ») : une isolation dans un espace de variables ne l'exprime pas, un arbre supervisé oui. D'autre part, le modèle prévient *à l'avance* : 100 % des courses frauduleuses sont signalées alors qu'il reste au moins 3 positions à parcourir, contre 75 % pour la règle. Une alerte qui tombe en même temps que la livraison n'est pas une détection, c'est une autopsie.

3.5 Le couplage avec la cybersécurité [F]

Le détecteur signale ; il ne prouve rien. Un signalement est une opinion tant que rien ne permet de trancher. Le second domaine (cybersécurité) fournit l'autre moitié :

- **Preuve de livraison scellée.** Passer une livraison à *livrée* exige une position GPS fraîche à moins de 500 m de la destination, puis scelle l'événement sous un HMAC-SHA256 dont la clé n'existe que côté serveur. Un simple SHA-256, qui est l'endroit où ce travail a commencé, ne prouve rien : les deux parties connaissent les champs scellés et peuvent recalculer l'empreinte, et la partie qui conteste la livraison est précisément celle qui le ferait. Sous HMAC, la contestation devient vérifiable.
- **Clé dérivée, pas réutilisée.** La clé de scellement est dérivée du secret racine (`HMAC(racine, "swiftdeliver/delivery-proof/v1")`), donc distincte de la clé de signature des JWT. Un secret unique pour deux usages est un défaut facile à manquer.
- **Vérification par le produit.** `GET /api/v1/admin/deliveries/{id}/proof` recalcule le sceau depuis la ligne stockée et répond `verified` ; 409 si la livraison n'a jamais été scellée. Un sceau que personne ne peut revérifier n'est qu'une chaîne de caractères.
- **Correction d'un IDOR.** N'importe quel client authentifié pouvait suivre la livraison d'un autre en devinant son identifiant (`GET /client/deliveries/{id}/location`). Le contrôle de propriété est désormais fait, et « ce n'est pas la vôtre » répond la même chose que « elle n'existe pas », pour ne pas offrir un oracle d'existence.

C'est ce couplage qui change l'usage : le modèle détecte une fraude *pendant* qu'elle se prépare, et le sceau permet de la *trancher* après coup. Sans le second, le premier n'est qu'un tableau de bord ; sans le premier, le second ne se déclenche qu'une fois le litige né.

3.6 Résultats vérifiés

- Tests backend : **63/63** (`cd backend && ./mvnw test`), dont 9 sur le sceau, 8 sur la révocation de jeton, 9 sur la limitation de débit et 4 sur les erreurs de requête.
- Détection en direct : **7/7** (`node verify_fraud_live.js`) — une trajectoire honnête ferme sans alerte, la même livraison avec un saut impossible déclenche une alerte. Les deux verdicts doivent différer : un détecteur bloqué à 0 échoue le second, bloqué à 1 il échoue le premier.
- Panneau de fraude : **14/14** (`node verify_fraud_panel.js`), qui vérifie aussi que le panneau déclare ses données d'entraînement comme synthétiques.
- Preuve de livraison : **23/23** (`node verify_proof.js`) — refus à 3 km, refus sans position, scellement à la porte via le même chemin que l'interface, vérification du sceau après un aller-retour en base, et deux livraisons au même endroit qui ne partagent pas le même sceau.
- Durcissement de l'authentification : **11/11** (`node verify_auth_hardening.js`) — le même jeton est accepté avant la déconnexion, refusé après, et un jeton neuf fonctionne ; cinq mauvais mots de passe répondent 401 et le sixième répond 429 avec un `Retry-After`. Les deux sens sont testés pour qu'aucune réponse constante ne passe.

4 Acquis, prévu, et limites assumées

Point	État	Statut
IA dans le produit	Forêt aléatoire entraînée hors ligne, évaluée à chaque position dans la JVM, alertes et courbe visibles.	F
Preuve de livraison HMAC	Seuil de distance, clé dérivée, sceau vérifiable par l'API.	F
Correction IDOR	Position d'autrui inaccessible.	F
Erreurs client en 500	Corrigé : corps illisible, paramètre manquant et mauvais type de chemin renvoient 400.	F
Limitation de débit sur le login	Contre la force brute : 5 essais par compte et 50 par adresse sur 15 minutes, compteur en mémoire remis à zéro par un succès. Reset au redémarrage et par instance : compromis assumé pour la mono-instance gratuite, pas une omission. Vérifié par <code>verify_auth_hardening.js</code> et par la CI.	F
Révocation de session (<code>tokenVersion</code>)	La déconnexion incrémente une version par compte, embarquée dans le JWT (claim <code>tv</code>) et vérifiée à chaque requête : un jeton volé meurt à la déconnexion, sur tous les appareils. Un jeton antérieur sans claim est lu comme version 0, pour ne pas déconnecter tout le monde lors de la montée de version. Vérifié par <code>verify_auth_hardening.js</code> et par la CI.	F
Code de remise (OTP)	Confirmation par le client en plus du livreur. Non implémenté.	P
Chaînage des sceaux	Un chaînage global exige un ordre total et un point d'ancrage ; en concurrence, il peut se ramifier. Un sceau HMAC honnête vaut mieux qu'un chaînage sur-affirmé. Reporté en connaissance de cause.	P
Données d'entraînement	Synthétiques : aucun historique réel de livreur n'existe. Les scores mesurent la validité du pipeline, pas la performance terrain. Le modèle et l'interface le déclarent.	L
GPS déclaratif	La position vient du téléphone. La démo assume une simulation.	L
OSRM sur le Cloud	Le serveur public de démonstration, partagé, pas une instance dédiée.	L
Offre gratuite	Base supprimée 30 jours après création, service mis en veille après inactivité. Documenté dans <code>DEPLOY.md</code> .	L
RLS PostgreSQL	Hors périmètre : incompatible avec le pool JPA mono-utilisateur sans refonte.	L

5 Protocole de mesure et reproductibilité

Toutes les valeurs citées sont réimprimées par un script, sur un jeu de test séparé du jeu d'entraînement. Le seuil d'un détecteur est choisi *du plus permissif vers le plus strict*, en s'arrêtant au premier qui respecte le budget de 2 % de fausses alertes : aucun détecteur n'est jugé sur un seuil choisi à la main pour le flatter. C'est ce qui rend la comparaison défendable.

- `python ml/supervised_vs_unsupervised.py` — imprime le tableau des quatre détecteurs et le détail par scénario (dont *drift_then_vanish* et le délai d'alerte). C'est la preuve du rejet de l'Isolation Forest.
- `python ml/train_fraud_model.py` — réentraîne et réécrit le modèle et les 200 vecteurs de parité.
- `cd backend && ./mvnw test` — 63 tests, dont la parité Java/scikit-learn, le sceau, la révocation de jeton, la limitation de débit et les erreurs de requête.

- `node verify_fraud_live.js, verify_fraud_panel.js, verify_proof.js, verify_auth_hardening.js, verify_pages.js, verify_assign.js, verify_stats.js` — vérifications contre la pile en marche.
- `python scripts/check_render_blueprint.py render.yaml` — cohérence du blueprint Cloud.

Conclusion

Le socle demandé est livré et éprouvé par des tests, pas par des intentions. L'innovation n'est pas un cahier hors ligne : c'est un modèle entraîné, embarqué dans le produit, qui note chaque position et dont le rejet de l'approche non supervisée est lui-même documenté. La partie cybersécurité ne sert pas de décor : elle transforme une alerte en preuve. Et les lignes qui ne sont pas faites sont écrites comme telles, avec ce qu'elles coûtent.